



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

ELEKTRO- UND
INFORMATIONSTECHNIK

PRAKTIKUM INFORMATIK 1

M. Niemetz

Kontrollstrukturen und Funktionen

Der torkelnde Läufer

Oktober 2025, Version: 1.1

Fakultät Elektro- und Informationstechnik

1 Einleitung

In dieser Aufgabe werden Sie zum ersten mal eigenständig ein (einfaches) Programm entwerfen und den Code für die Umsetzung Ihrer Strategie schreiben.

Dafür benötigen Sie Kontrollstrukturen, speziell die Schleifen und Verzweigungen, aber auch Funktionen mit Parameterübergabe. Außerdem lernen Sie das Arbeiten mit Arrays und Präprozessorgroößen kennen.

Die folgenden Themen sollten Sie nachlesen falls Sie sie nicht mehr ausreichend präsent haben, oder sie noch nicht ausreichend detailliert in der Vorlesung besprochen wurden:

Vorbereitung

- ☐ ☕ **Präprozessorkonstanten:** Lesen Sie nach, wie Sie mit dem Präprozessor Konstanten definieren können. 3
- ☐ ☕ **Arrays:** Lesen Sie nach, wie Sie ein- und mehrdimensionale Arrays anlegen und verwenden können. 4
- ☐ ☕ **Schleifen:** Lesen Sie nach, wie Sie die verschiedenen verfügbaren Schleifenvarianten in C einsetzen können. 4
- ☐ ☕ **Modulo-Operator:** Machen Sie sich klar, wie der Modulo-Operator arbeitet. 6

2 Die Aufgabe

Bei digitalen Schlüsseln (zum Beispiel bei ssh zum Fernbedienen von Rechnern, bei openssl zum Erstellen von Schlüsseln für den sicheren HTTP-Verkehr oder bei gnupg für verschlüsselte Email-Kommunikation) ist es notwendig zu überprüfen, ob auch der korrekte Schlüssel eingesetzt wird.

Für diesen Abgleich werden sogenannte Schlüssel-Fingerabdrücke (fingerprint) erstellt und verglichen, die deutlich kompakter sind, als die Schlüssel mit ihren typischerweise 1–4 kByte. Allerdings sind die Vergleiche dieser Werte immer noch sehr unergonomisch.

Hier ein Beispiel für einen Fingerabdruck eines gnupg-Schlüssels:
Vergleiche 2513 9B2D 90AF 20EE B897 24A0 8E7F 00D6 11C9 086F
mit 2513 B92D 90AF 20EE 8B97 24A0 8F7E 00D6 11C9 068F

Um den visuellen Vergleich zu erleichtern, wurden die sogenannten *random art*-Diagramme entwickelt [1].

```

+-----+
|      . . . + . |
|      o o + |
|      o + . o |
|      = ++ * |
|      . . + = o E |
|      . . o = . B . |
|      + . + = . * . O |
|      . . o B . + o |
|      . + . o + . |
+-----+

```

Abbildung 1: Beispiel für ein von ssh generiertes *random-art*-Diagramm. Die unterschiedlichen Zeichen sollen dabei unterschiedliche Besuchshäufigkeiten des Läufers repräsentieren.

Hierfür wird die Kette aus Bits, aus denen der Fingerprint-Wert besteht, in der folgenden Art in ein Diagramm umgewandelt (siehe Abbildung 1).

Das Bild entsteht, indem eine Figur Züge durchführt, ähnlich denen eines Läufers beim Schach (engl.: bishop). Die Figur zieht immer um ein Feld weiter, worauf die Bewegungsrichtung aufgrund der jeweils nächsten zwei Bit neu ausgewählt wird. Der Läufer torkelt deswegen über das Spielfeld (engl.: drunken bishop), wobei bei jedem Feld des Spielfeldes mitgezählt wird, wie häufig die Figur darauf zu stehen kommt. Diese Besuchshäufigkeit wird dann in der Grafik über verschiedene Zeichen dargestellt.

Ihr Ziel in dieser Aufgabe ist es, solche Diagramme zu berechnen. Dafür gehen Sie in mehreren Schritten vor.

2.1 Ausgeben des Bildes

Bevor Sie mit der eigentlichen Berechnung beginnen, ist es hilfreich, eine Funktion zur Ausgabe des Ergebnisses vorzubereiten, um das Testen bei der Entwicklung zu vereinfachen. Die Ausgabe soll analog zur Darstellung in Abbildung 1 gestaltet werden. In den nachfolgenden Absätzen wird diese Ausgabe schrittweise entwickelt.

Zunächst muss die Größe des Spielfeldes festgelegt werden. Da diese Werte an den verschiedensten Stellen des Programms benötigt werden und an diesen Stellen auch richtig zusammenpassen müssen, verwenden Sie Präprozessorkonstanten für Breite und Höhe des Spielfeldes.

Grundsätzlich definieren Sie Präprozessorkonstanten folgendermaßen:

```
#define MY_BORING_CONSTANT 42
```

Die Konvention gibt dabei vor, ausschließlich Großbuchstaben für den Namen zu verwenden.



Schreiben Sie eine Funktion, die das noch leere Feld auf dem Bildschirm ausgibt.

Recherche ☕

Präprozessorkonstanten

Hinweis: Da C keine Umlaute im Programmtext erlaubt, ist es doppelt sinnvoll, alle Variablen mit englischsprachigen Bezeichnern zu definieren.

Beachten Sie auch, dass in einer Präprozessorzeile keine anderen Dinge stehen dürfen und auch kein Semikolon am Ende angefügt wird.

Der Grund ist, dass der Präprozessor eine rein textuelle Modifikation in Form einer Suchen-und-Ersetzen-Operation vornimmt – der Präprozessor kann kein C wie Sie oder der Compiler.

Aus Gründen der Lesbarkeit und um Missverständnisse zu vermeiden, sollten alle Präprozessorkonstanten am Anfang des Programms definiert werden, da der Präprozessor den Programmtext konsequent von oben nach unten abarbeitet – ungeachtet der Strukturierung in Funktionen oder die spätere Reihenfolge der Ausführung.

Definieren Sie zwei Konstanten für `HEIGHT` und `WIDTH` mit dem Wert 7 bzw. 15.

Nun wird eine Variable benötigt, die das „Spielfeld“, genauer gesagt die Häufigkeit, mit der der Läufer auf jedem einzelnen Feld des Spielfeldes zum Stehen kommt, gespeichert wird. Hierfür verwenden Sie ein Array und – da das Spielfeld zweidimensional ist – eines mit zwei Indizes, welche Breite und Höhe des gewünschten Spielfelds bezeichnen:

```
int Board[SOME_X_DIMENSION][SOME_Y_DIMENSION]={0};
```

Nun können Sie eine Funktion `printBoard` ohne Parameter schreiben, welche das Spielfeld (eine globale Variable) ausdrückt. Dafür setzen Sie verschiedene Schleifen ein, die verschachtelt und/oder hintereinander ausgeführt werden.

Hinweis: Zweidimensionale Arrays können Sie beim Zugriff analog zu den bereits bekannten eindimensionalen verwenden, also zum Beispiel:

```
Board[x][y]+=1;
```

Wichtig: Beachten Sie, dass die Elemente in einem Array in jeder Dimension mit Null beginnend durchnummeriert werden. Damit lautet das letzte existierende Element (also dasjenige rechts unten) in diesem Fall

```
Board[SOME_X_DIMENSION-1][SOME_Y_DIMENSION-1].
```

Für die Felder des Spielfeldes müssen Sie die im jeweiligen Element des Arrays gespeicherte Häufigkeit in ein Zeichen kodieren. Dafür definieren Sie ein Array mit dem gewünschten „Zeichensatz“ und eine Präprozessorkonstante für `MAPLEN` mit dem Wert 15.

```
unsigned char Mapping[MAPLEN]=
{' ','.',',','o','+', '=', '*', 'B', '0', 'X', '@', '%', '&', '#', '/', '^'};
```

Dann können Sie eine Häufigkeit einfach folgendermaßen auf dem Bildschirm als Zeichen codiert ausgeben:

Recherche ☕
Arrays

Recherche ☕
Schleifen

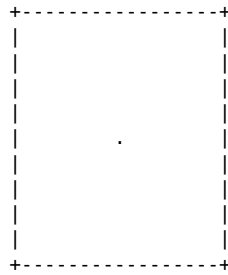


Abbildung 2: Beispiel für ein frisch initialisiertes Spielfeld mit dem Läufer in der Ausgangsposition.

```
unsigned int a = 5; // Beispiel für Häufigkeitswert
printf("%c", Mapping[ a % MAPLEN]);
```

Frage: Wie funktioniert dieser Programmcode? Vor allem: Wozu dient das Operatorzeichen zwischen `a` und `MAPLEN`?

Testen Sie Ihre Funktion, ob sie für unterschiedliche Werte für Breite und Höhe des Spielfeldes korrekt funktioniert. Schreiben Sie testweise auch unterschiedliche Häufigkeitswerte in die `Board`-Variable um die Umsetzung in die verschiedenen Zeichen zu testen.

Hinweis: Speichern Sie die Ausgaben bei den Tests zur Dokumentation ab, indem Sie sie in eine Datei kopieren.



Testen Sie Ihre Funktion `printBoard` ausgiebig.

2.2 Initialisierung des Spielfeldes

Nun gilt es, den Weg des Läufers zu berechnen. Hierfür erstellen Sie zwei globale Variablen, welche jeweils die aktuelle x- bzw. y-Achsenposition speichern.

Schreiben Sie eine Funktion zur Initialisierung des Spielfeldes, welche folgende Aufgaben wahrnimmt:

- Füllen aller Felder der Variable `Board` mit der Wert 0;
- Setzen der initialen Läuferposition auf die Spielfeldmitte;
- Erhöhen desjenigen Feldes des Spielfeldes an dem sich aktuell der Läufer befindet um eins;

Gibt man danach das Spielfeld mit der bereits implementierten Funktion aus, erhält man ein Bild wie etwa das in Abbildung 2 gezeigte.



Schreiben Sie eine Funktion zur Initialisierung des Spielfeldes.

2.3 Das Torkeln des Läufers

Nun kann der Läufer beginnen, sich über das Feld zu bewegen. Dafür schreiben Sie eine Funktion, welche einen Parameter des Typs `integer` erhält, der eine von vier möglichen Bewegungsrichtungen als Wert von 0 bis 3 angibt:



Implementieren Sie eine Funktion zur Bewegung des Läufers.

Richtung 0: Der Läufer bewegt sich nach links oben, d.h. die x- und die y-Koordinate des Läufers werden beide um eins im Wert reduziert.

Richtung 1: Der Läufer bewegt sich nach rechts oben, d.h. die x-Koordinate wird inkrementiert, die y-Koordinate dekrementiert.

Richtung 2: Der Läufer bewegt sich nach links unten, d.h. die x-Koordinate wird dekrementiert, die y-Koordinate inkrementiert.

Richtung 3: Der Läufer bewegt sich nach rechts unten, d.h. die x- und die y-Koordinate des Läufers werden beide um eins im Wert erhöht.

Falls dieser Bewegungsschritt dazu führt, dass die Grenzen des Spielfeldes überschritten würden, entfällt die Bewegung des Läufers in diejenige Richtungskomponente in der die Wand die Bewegung blockiert.

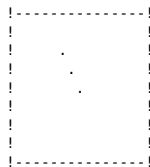
Hinweis: Dies lässt sich sehr gut nach der Ausführung des Schritts durch eine geeignete Limitierung der x- und y-Position des Läufers umsetzen.

Nun muss der Schritt des Läufers noch auf dem Spielfeld verbucht werden. Hierfür ist in der Variable **Board** das Feld mit den aktuellen Koordinaten des Läufers zu inkrementieren.

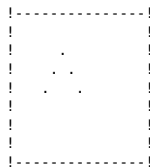
Testen Sie Ihre Funktion zur Bewegung des Läufers, indem Sie verschiedene Schritte ausführen und das Spielfeld jeweils zwischendurch ausgeben lassen.

Folgende Sequenz wäre ein Beispiel:

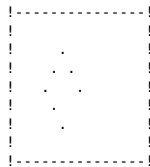
Zwei Mal Richtung 0



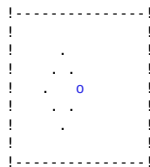
dann zwei Mal Richtung 2



dann zwei Mal Richtung 3

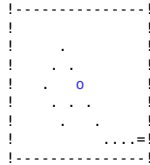


dann zwei Mal Richtung 1



Testen Sie Ihre Entwicklung!

dann zehn Mal Richtung 3



2.4 Eine längere Läufer tour

Um nun eine längere Tour des Läufers zu bilden, muss z.B. eine lange Integerzahl in Einzelschritte zerlegt werden.

Schreiben Sie eine Funktion mit einem Parameter des Typs `unsigned long long int`, die folgende Aufgaben erledigt:

1. Ermitteln der Anzahl an Bytes des Parameters der Funktion mittels `sizeof`;
2. Ausführen einer Schleife für alle im Parameter enthaltenen Läuferschritte; da ein Schritt aus zwei Bits und ein Byte andererseits aus acht Bits besteht, fallen für jedes Byte des Parameters vier Läuferschritte an.
 - a) Berechnen des aktuellen Parameterwerts modulo¹ vier um den Wert der beiden niederwertigsten Bits zu isolieren;² Durch diese Berechnung erhalten Sie einen Wert im Bereich von null bis drei (also eine zwei Bit lange Zahl), den Sie als Zwischenergebnis in einer Variable speichern und im nächsten Schritt zur Ausführung eines Läuferschritts verwenden können.
 - b) Ausführen des Einzelschritts der durch diese beiden Bits spezifiziert wird durch Aufrufen der Läuferfunktion aus dem vorangehenden Abschnitt und übergeben des in der vorangehenden Berechnung erhaltenen Ergebnisses;
 - c) Setzen des Parameterwerts der Funktion auf (abgerundet) ein Viertel des bisherigen Werts, um die beiden verarbeiteten niederwertigsten Bits abzuschneiden; Hierbei ist es praktisch, dass in C bei einer Division zweier Ganzzahl-Werte automatisch die evtl. entstehenden Nachkommastellen abgeschnitten werden.³

Frage: Sie lesen hier wie gewöhnlich den Parameter der Funktion – aber Sie beschreiben ihn auch, um den Wert zu ändern. Wie stellen



Schreiben Sie eine Funktion zum Zerlegen einer Ganzzahl in Einzelschritte.

Recherche ☕

Modulo-Operator

¹Die Modulo-Berechnung ermittelt den Divisionsrest bei einer Division zweier ganzer Zahlen. Mathematisch ist das gewissermaßen der Nachkomma-Anteil. Der Operator wird in C durch das Prozentzeichen dargestellt.

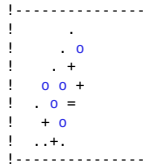
²Falls Sie bereits bitweise Operatoren beherrschen, können Sie für das Isolieren der beiden niederwertigsten Bits natürlich auch ein bitweises AND verwenden.

³Auch hier können Sie alternativ auch mit einer bitweisen Schiebeoperation arbeiten, falls Sie diese schon kennen und hier einsetzen wollen.

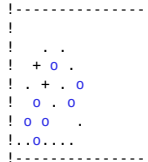
Sie sich vor, dass das funktioniert? Bemerkte dabei der aufrufende Programmteil, dass der übergebene Wert geändert wird?

Testen Sie nun Ihre Funktion mit den beiden folgenden Werten:

0x0e2f8bac75342a72cULL



0x0e2f8bca75842af2cULL



Wichtig: Verwenden Sie für die Literalkonstanten das Suffix `ULL`, um dem Compiler mitzuteilen, dass es sich bei dem Wert um eine Konstante handelt, die als Wert des Typs `unsigned long long int` zu behandeln ist.

Hinweis: Falls Ihr C-Compiler den hier verwendeten Datentyp `unsigned long long int` mit einer anderen Länge als 8 Byte implementiert, erhalten Sie ein anderes Ergebnis.

Literatur

- [1] Dirk Loss, Tobias Limmert, Alexander von Gernler: *The Drunken Bishop: An Analysis of the OpenSSH Fingerprint Visualization Algorithm*, 2009